ResearchIQ

MASTER INTEGRATION PLAN

Public & Private Library • SMTP OTP • RBAC • Research Assistant • Insights • Secure RAG

Project	ResearchIQ - Intelligent Research Document Analysis and Retrieval
Implementation base	Fork and extend the existing RAG Web UI modular monolith
Database baseline	Existing MySQL ragwebui schema inspected on 15 September 2026
Version	1.0 - Codex-ready master implementation specification

ARCHITECTURE INVARIANT
Do not build another RAG system. Reuse the working ingestion, parsing, chunking, embeddings, Chroma retrieval, MinIO storage, chat, streaming and citation foundations. ResearchIQ is the extended product built on top of that baseline.

Prepared: 15 September 2026

Contents

1. Executive Summary
2. Baseline System and Architecture Rules
3. Verified Current MySQL Schema
4. Target ResearchIQ Architecture
5. Authentication - SMTP OTP
6. Authorization - Researcher/Admin RBAC
7. Public Library and Institutional Corpus
8. Private Library
9. Academic Metadata, Departments and Bookmarks
10. Search, Filters and Catalog APIs
11. Document Ingestion and Processing
12. Research Assistant and Source Scopes
13. Secure Retrieval and Citation Integrity
14. Insights Tab - Document Discovery and Selection
15. Compare Research
16. Analyze Research
17. Contradiction Analysis
18. Candidate Research Gaps
19. Analysis Persistence and History
20. Modular Code Structure
21. Security, Safety and Abuse Resistance
22. Reliability, Performance and Scalability
23. Alembic Migration Strategy
24. End-to-End Implementation Phases
25. Acceptance and Evaluation Criteria
26. Codex Execution Contract
Appendix A. Proposed Schema
Appendix B. Core API Contracts
Appendix C. Threat and Failure Checklist

1. Executive Summary

ResearchIQ will evolve the existing working RAG Web UI into a UAF-focused research intelligence platform. The implementation will be incremental: preserve proven RAG infrastructure, add secure identity and authorization, introduce Public and Private Library semantics, attach academic metadata, and then build evidence-grounded research workflows such as Compare, Analyze, Contradictions and Candidate Research Gaps.

Core product principle
The project contribution is not a second vector database or another chatbot. It is the domain layer that turns generic RAG into a secure university research workspace with institutional knowledge, personal research, structured analyses and traceable evidence.

1.1 Final capability set

- SMTP OTP authentication with secure session handling.
- Researcher/Admin RBAC with explicit professor email allowlisting; no Google Auth in the current scope.
- A system-owned Public Library containing administrator-curated institutional research.
- User-owned Private Libraries that preserve the existing knowledge-base ownership model.
- Academic metadata, departments, filters and bookmarks without duplicating source files.
- Research Assistant with Public, Private, Both and Selected Documents retrieval scopes.
- Insights tab with structured Compare and Analyze workflows.
- Evidence-backed contradiction analysis and corpus-qualified Candidate Research Gaps.
- Persistent analysis history and citation/evidence snapshots.
- Security controls for OTP abuse, unauthorized document access, prompt injection, malicious uploads and hallucinated citations.

1.2 Explicitly out of scope for this phase

- Google OAuth/OIDC.
- A second RAG backend or second database.
- GraphRAG, microservices, Kubernetes, Redis/Celery unless later performance evidence justifies them.
- Automatic admin status for all university email accounts.
- Definitive scientific claims that a research gap exists outside the indexed corpus.

2. Baseline System and Architecture Rules

The working baseline already provides the difficult RAG foundation. It should be treated as a protected platform, not replaced.

Existing capability	Decision
PDF/DOCX/Markdown/Text ingestion	Reuse
Async document processing	Reuse
MinIO object storage	Reuse
Document chunking	Reuse
Embedding generation	Reuse
Chroma vector retrieval	Reuse

Existing capability	Decision
LLM provider integration	Reuse
Streaming chat	Reuse
Citation rendering	Reuse and harden
MySQL relational data	Extend through Alembic
Next.js frontend	Rebrand and extend
FastAPI backend	Extend as a modular monolith

2.1 Non-negotiable implementation rules

- Feature -> API -> service -> RAG/DB/LLM. Avoid giant route files and duplicated retrieval code.
- Authorization must be enforced server-side before private evidence reaches the LLM.
- Different prompts do not imply different RAG systems. One ingestion/indexing foundation powers multiple workflows.
- LLM output must be structured data for application rendering. The model must not generate authorization logic or frontend HTML.
- All schema changes go through Alembic migrations.
- After every major phase, run tests and verify the end-to-end path before proceeding.

3. Verified Current MySQL Schema

The following tables and key columns were inspected directly from the active ragwebui MySQL database. The master plan extends this schema instead of inventing a replacement.

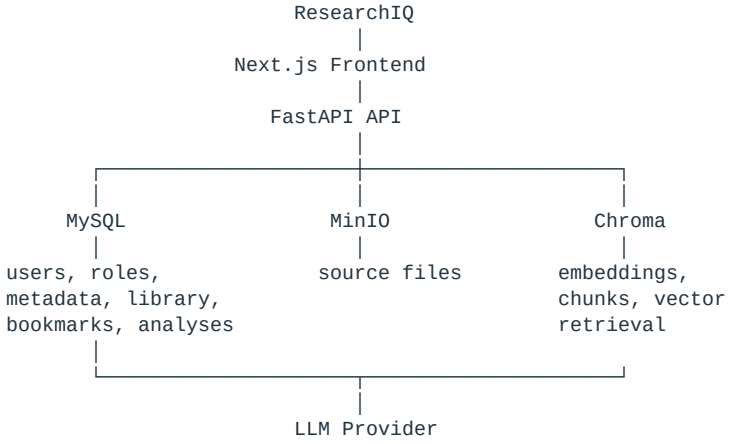
Table	Current purpose / key fields
users	id, email, username, hashed_password, is_active, is_superuser, timestamps
knowledge_bases	id, name, description, user_id, timestamps
documents	id, file_path, file_name, file_size, content_type, file_hash, knowledge_base_id, timestamps
document_uploads	upload staging and processing state
document_chunks	id, kb_id, file_name, hash, document_id, chunk_metadata JSON
processing_tasks	document processing task state and errors
chats	id, title, user_id, timestamps
messages	chat messages
chat_knowledge_bases	many-to-many link between chats and knowledge bases
api_keys	user API keys
alembic_version	migration state

3.1 Verified ownership relationships

```
users
  ├── knowledge_bases
  │     ├── documents
  │     │     └── document_chunks
  │     ├── document_uploads
  │     └── processing_tasks
  ├── chats
  │     ├── messages
  │     └── chat_knowledge_bases -> knowledge_bases
  └── api_keys
```

Important schema finding
knowledge_bases.user_id is required, so the institutional Public Library should not be personally owned by a professor. Use a dedicated non-login system owner, while professor admins receive management permission through RBAC.

4. Target ResearchIQ Architecture



4.1 Functional modules

- Authentication
- Home Dashboard
- Library - Public / Private
- Research Assistant
- Insights - Compare / Analyze
- Bookmarks
- Research History
- Admin Corpus Management
- Settings
- Help & Support

5. Authentication - SMTP OTP

Google authentication is excluded from the current plan. ResearchIQ will use email verification through SMTP-delivered one-time passwords and then establish a normal authenticated application session.

```
Enter email -> Request OTP -> Generate secure OTP -> SMTP delivery
-> Verify OTP -> Resolve/create user -> Resolve role -> Secure session
```

5.1 Required database changes

Change	Purpose
users.hashed_password -> nullable	OTP-first users may never create a local password; retain the column for optional future password login.
email_otps table	Stores hashed OTP state, purpose, expiry, attempts and one-time-use status.

5.2 OTP security requirements

- Cryptographically secure random generation.
- Store only an OTP hash, never plaintext.
- 5-10 minute expiration.
- One-time use; mark used immediately after successful verification.
- Maximum verification attempts.
- Resend cooldown.
- Rate limit by email and IP.
- Issuing a new OTP invalidates the previous active OTP.
- Use TLS for SMTP.
- Keep SMTP credentials only in backend secrets/environment.
- Never log plaintext OTP values.
- Return generic responses that avoid easy account enumeration.

5.3 Session model

OTP verifies identity only at login. After successful verification the application should create an authenticated session using Secure, HttpOnly and SameSite cookie settings. Do not require an OTP for every API request and do not store long-lived session secrets in localStorage.

5.4 Suggested endpoints

```
POST /api/v1/auth/request-otp
POST /api/v1/auth/verify-otp
POST /api/v1/auth/logout
GET  /api/v1/auth/me
POST /api/v1/auth/resend-otp   # optional convenience endpoint
```

6. Authorization - Researcher/Admin RBAC

Authentication answers who the user is. ResearchIQ authorization decides what the user is allowed to do.

6.1 Roles

```
roles
  researcher
  admin

user_roles
  user_id
  role_id
```

Keep users.is_superuser for exceptional system-level administration. A professor content administrator should normally have role=admin and is_superuser=false.

6.2 Admin allowlist

```
admin_allowlist
  id
  email
  department_id
  is_active
  added_by
  created_at
  updated_at
```

Professors are authorized by exact normalized email entries. A university email domain alone must never grant admin rights.

6.3 Permission matrix

Capability	Researcher	Admin
View/search/filter Public Library	Yes	Yes
Bookmark public documents	Yes	Yes
Query/Compare/Analyze public documents	Yes	Yes
Upload to Public Library	No	Yes
Edit public academic metadata	No	Yes
Publish/archive/delete public documents	No	Yes
Upload to own Private Library	Yes	Yes
Access another user private library	No	No by default
Manage system configuration	No	Only superuser where applicable

Privacy rule
Admin access to institutional public content does not automatically imply surveillance access to private user libraries. Keep those permissions separate unless a future governance policy explicitly changes them.

7. Public Library and Institutional Corpus

7.1 Extend knowledge_bases

```
knowledge_bases
  existing fields...
  scope             # private | public
  is_system_managed # boolean
```

7.2 System-owned institutional knowledge base

```
ResearchIQ System User (non-login)
        ↓
UAF Research Corpus
  scope = public
  is_system_managed = true
        ↓
public documents
```

The system account exists to satisfy the existing non-null knowledge_bases.user_id relationship while preventing institutional ownership from being tied to any professor account. It must not be able to authenticate interactively.

7.3 Public researcher capabilities

- View documents.
- Search by title/author/topic.
- Filter by department, type, year/date and other supported metadata.
- Bookmark documents.
- Use public sources in Research Assistant.
- Select documents for Compare and Analyze.

7.4 Admin corpus capabilities

- Upload research documents.
- Enter/edit academic metadata.
- Assign department and type.
- Monitor processing.
- Retry failed ingestion.
- Publish/unpublish or archive.
- Delete through a safe lifecycle that cleans relational, object-storage and vector state.

8. Private Library

Private Library should reuse the current user-owned knowledge-base model rather than create a parallel storage design.

```
Current user
   ↓
private knowledge base
  scope = private
  user_id = current_user.id
   ↓
user documents
```

8.1 User actions

- Upload supported documents.
- Track Pending / Processing / Ready / Failed states.
- Search and filter personal documents.
- Query or summarize personal documents.
- Select personal documents for Compare or Analyze.
- Delete own documents safely.

8.2 Access rule

A private document is permitted only when its knowledge base is private and the knowledge base user_id equals the authenticated user. Every backend endpoint that accepts document IDs must re-check this relationship.

9. Academic Metadata, Departments and Bookmarks

9.1 Keep documents technical

The existing documents table should stay focused on file identity/storage/indexing fields. Academic semantics should be attached through a one-to-one metadata table.

```
academic_document_metadata
  id
```

```
  document_id
  title
  authors_text
  abstract
  department_id
  document_type
  publication_year
  publication_date
  supervisor
  keywords
  doi
  language
  created_at
  updated_at
```

9.2 Departments

```
departments
  id
  name
  code
  is_active
  created_at
  updated_at
```

Use normalized department IDs rather than inconsistent strings such as "CS", "Comp Sci" and "Computer Science".

9.3 Bookmarks

```
bookmarks
  id
  user_id
  document_id
  created_at
  UNIQUE(user_id, document_id)
```

Bookmarking stores a user-to-document reference only. It must not duplicate PDFs, document rows or vector chunks.

10. Search, Filters and Catalog APIs

Document discovery is primarily a relational catalog problem, not an LLM problem.

10.1 Supported filters

- scope = public | private | both
- free-text title/keyword search
- department
- document type
- author
- publication year/date range
- processing status
- pagination and sort

10.2 Example catalog API

```
GET /api/v1/documents
  ?scope=public|private|both
  &search=
  &department_id=
  &document_type=
  &author=
  &year_from=
  &year_to=
  &page=
  &page_size=
```

10.3 Query discipline

Use SQL for catalog filtering, pagination and ownership checks. Use vector retrieval for semantic evidence search after the application knows which documents or scopes the user is authorized to access.

11. Document Ingestion and Processing

Public and Private Library uploads reuse the existing RAG Web UI ingestion pipeline.

```
Upload
  ↓
document_uploads
  ↓
processing_tasks
  ↓
MinIO
  ↓
parser / text extraction
  ↓
chunking
  ↓
embedding
  ↓
Chroma
  ↓
documents + document_chunks
  ↓
Ready
```

11.1 Public admin upload

Admin uploads should collect academic metadata at upload time or immediately after ingestion, then publish only after processing reaches Ready. Normal researchers never receive the Public Library upload control.

11.2 Processing states

- Pending
- Queued
- Processing
- Ready
- Failed
- Duplicate

Do not show fabricated progress percentages unless the backend measures actual stage progress.

12. Research Assistant and Source Scopes

12.1 Supported source modes

- Public
- Private
- Both
- Selected Documents

12.2 Flow

```
User question
   ↓
resolve authenticated user and requested scope
   ↓
compute permitted KB/document set
   ↓
```

```
retrieve evidence only from that set
   ↓
assign evidence IDs
   ↓
LLM synthesis
   ↓
validate evidence references
   ↓
render answer + citations
```

12.3 Existing chat reuse

The existing chats table already belongs to users and chat_knowledge_bases already links a chat to multiple knowledge bases. Preserve this foundation. Add more precise selected-document scoping only where Compare, Summarize or Analyze requires it.

13. Secure Retrieval and Citation Integrity

13.1 Authorization before retrieval

> **Do not post-filter sensitive evidence**
> Never retrieve another user's private chunks, send them to the LLM, and then hide the citation afterward. The authorized document set must constrain retrieval before the model receives context.

13.2 Chunk metadata

```
Public chunk metadata example:
{
  "document_id": 81,
  "knowledge_base_id": 1,
  "scope": "public",
  "department_id": 4,
  "document_type": "paper",
  "year": 2025,
  "page": 12
}

Private chunk metadata example:
{
  "document_id": 126,
  "knowledge_base_id": 44,
  "scope": "private",
  "owner_id": 17,
  "page": 28
}
```

The existing document_chunks.chunk_metadata JSON field can hold retrieval metadata, but authorization-critical truth must still be validated against trusted relational ownership.

13.3 Evidence IDs and deterministic citation mapping

```
E1 -> document_id=81, chunk_id=..., page=12, excerpt=...
E2 -> document_id=81, chunk_id=..., page=15, excerpt=...
E3 -> document_id=96, chunk_id=..., page=7, excerpt=...
```

The LLM references E1/E2/E3. The backend maps those IDs to title, author, department, year and page from authoritative application data. The model must never invent bibliography metadata.

13.4 Insufficient evidence

When retrieved evidence is weak, missing or conflicting, ResearchIQ should explicitly report insufficient evidence instead of fabricating a confident answer.

14. Insights Tab - Document Discovery and Selection

Insights combines normal catalog discovery with controlled research workflows.

```
Search documents
Filters: Public / Private / Both, Department, Type, Author, Date/Year

☐ Document A
☐ Document B
☐ Document C

[Compare]   [Analyze]
```

14.1 Selection rules

- Checkboxes submit real immutable document IDs, not filenames.
- Backend checks each document exists.
- Backend checks processing status is Ready.
- Backend checks the current user may access each selected document.
- Unauthorized IDs are rejected even if manually crafted in the request.

14.2 Scope behavior

- Public: institutional documents only.
- Private: current user private documents only.
- Both: union of authorized public + current-user private documents.

15. Compare Research

Compare is a document-scoped RAG workflow. It is not a second vector system and it is not merely a vague prompt.

```
Selected Paper A -> retrieve evidence ONLY from A -> Evidence A
Selected Paper B -> retrieve evidence ONLY from B -> Evidence B
                                  ↓
                          comparison service
                                  ↓
                          structured JSON
                                  ↓
                          React table UI
```

15.1 Comparison criteria

- Research objective/question
- Methodology
- Study area/context
- Population/sample
- Dataset
- Variables
- Models/techniques
- Main findings
- Limitations
- Conclusion
- Future work

15.2 Structured output

```
{
  "documents": [...],
  "criteria": [
    {
      "criterion": "Methodology",
```

```
        "values": [
          {"document_id": 81, "text": "...", "citations": ["E2"]},
          {"document_id": 96, "text": "...", "citations": ["E7"]}
        ]
      }
    ]
  }
```

The frontend renders the comparison table. The LLM should not generate HTML or CSS.

15.3 Cross-domain comparison

Do not hard-block interdisciplinary comparisons. Instead expose a compatibility indicator such as Highly related, Related, Weakly related or Different domains, with a warning when direct interpretation is limited.

16. Analyze Research

Analyze performs a deeper, multi-stage synthesis using selected documents plus carefully selected related corpus evidence.

16.1 Staged workflow

1. Validate selected document access and readiness.
2. Run document-scoped retrieval for each selected document.
3. Build a structured research profile per document: objective, methodology, dataset, findings, limitations, future work.
4. Derive topic and metadata signals from the selected set.
5. Use SQL filters to narrow the institutional corpus.
6. Use semantic retrieval to choose only the most relevant related documents/evidence.
7. Run cross-study synthesis over bounded evidence.
8. Return structured sections with citations.

16.2 Related-paper discovery

```
Example:
700 Agronomy documents
   ↓ SQL/metadata narrowing
80 potentially related
   ↓ semantic retrieval
Top 10 related documents
   ↓ evidence extraction
bounded evidence set for LLM analysis
```

16.3 Analyze outputs

- Overall summary
- Key insights
- Common findings
- Major limitations
- Methodological differences
- Contradictions
- Candidate Research Gaps
- Research trends
- Future directions

17. Contradiction Analysis

Contradictions should be modeled as evidence relationships rather than a binary "paper A is right / paper B is wrong" judgment.

```
{
  "claim": "...",
  "supporting": ["E1", "E2"],
  "contradicting": ["E7"],
  "qualifying": ["E9"],
  "interpretation": "..."
}
```

17.1 Interpretation factors

- different population/sample
- different location/environment
- different dataset
- different methodology
- different time period
- different sample size
- different measurement conditions

Scientific framing
Present results as Evidence Interpretation. The system may identify disagreement and plausible methodological reasons, but should not declare that one study is scientifically invalid.

18. Candidate Research Gaps

ResearchIQ should surface candidate gaps supported by the indexed corpus, not claim universal scientific gaps.

18.1 Supported gap categories

- Limited Evidence
- Conflicting Evidence
- Understudied Population
- Understudied Location
- Understudied Method
- Outdated Evidence
- Sparse Recent Evidence
- Limited Field Validation

18.2 Evidence rule

Every candidate gap must name the evidence pattern that produced it and link back to supporting sources. Example: most retrieved studies use controlled imagery, while evidence from uncontrolled field environments is sparse within the indexed corpus.

18.3 Required wording

Required qualification
Use language such as "candidate research gap within the indexed corpus" or "appears under-researched in the available evidence." Do not state that ResearchIQ has proven a definitive gap in the global literature.

19. Analysis Persistence and History

```
analysis_runs
```

```
  id
  user_id
  analysis_type
  status
  scope
  request_json
  result_json
  created_at
  completed_at
  error_message

analysis_documents
  analysis_id
  document_id
  role

analysis_evidence
  id
  analysis_id
  document_id
  chunk_id
  page
  excerpt
  retrieval_rank
  retrieval_score
  evidence_hash
```

Persisting evidence snapshots makes past analyses auditable even if the vector index is later rebuilt. Similarity score must never be presented as scientific confidence.

20. Modular Code Structure

20.1 Backend target

```
backend/app/
  api/
    auth.py
    library.py
    documents.py
    bookmarks.py
    research.py
    insights.py
    compare.py
    analyze.py
    admin.py

  services/
    auth/
      otp_service.py
      email_service.py
      session_service.py
    authorization/
      role_service.py
      document_access.py
    documents/
      catalog_service.py
      ingestion_service.py
    rag/
      retrieval.py
      document_scope.py
      citations.py
    research/
      summarize.py
      compare.py
      analyze.py
      contradictions.py
      candidate_gaps.py
```

20.2 Prompt modules

- qa_prompt
- summary_prompt
- comparison_prompt
- analysis_prompt
- contradiction_prompt
- candidate_gap_prompt

All prompt modules share the same retrieval and evidence infrastructure. Prompt specialization is workflow behavior, not a separate RAG deployment.

20.3 Frontend responsibilities

- Render structured JSON, never trust LLM-generated markup.
- Hide controls for UX, but rely on backend authorization for security.
- Preserve the existing RAG Web UI design system while introducing ResearchIQ navigation and research-specific states.
- Display explicit processing/loading/error states.
- Make citations inspectable and traceable to source/page/excerpt.

21. Security, Safety and Abuse Resistance

21.1 Authentication security

- OTP hashing, short expiry, attempt limits and single-use semantics.
- Rate limiting for request/resend/verify endpoints.
- Secure HttpOnly SameSite cookies.
- Session expiration and logout revocation.
- SMTP credentials kept out of source control and logs.

21.2 Authorization security

- Every protected route resolves the current user server-side.
- Never trust client-supplied owner_id, role or scope.
- Every selected document ID is re-authorized.
- Public management endpoints require Admin role.
- Private retrieval requires KB ownership by the current user.

21.3 Prompt injection defense

Treat retrieved document text as untrusted data. A PDF may contain instructions such as "ignore previous instructions" or "reveal secrets". Those strings must remain evidence text and must never change system policy, authorization, retrieval scope or backend actions.

21.4 Upload security

- Validate extension and MIME type.
- Enforce maximum size and non-empty file.
- Generate safe storage object names.
- Prevent path traversal.
- Handle malformed PDFs/documents safely.
- Detect duplicate file hashes where appropriate.
- Do not execute embedded content.

21.5 Output safety

- Sanitize rendered Markdown/HTML.
- LLM never decides permissions.
- LLM never invents page numbers/authors/titles.
- Require evidence IDs for grounded claims.
- Support an insufficient-evidence response path.

21.6 Security tests that must exist

- User A requests User B private document by ID.
- Researcher calls Admin upload/edit/delete endpoint.
- Expired/reused OTP verification.
- OTP brute-force/resend abuse.
- Document contains prompt-injection instructions.
- Malicious or oversized upload.
- Citation ID not present in evidence set.

22. Reliability, Performance and Scalability

22.1 Reliability

- Idempotent ingestion where feasible.
- Retryable processing failures with visible error state.
- Transactional updates for related relational records.
- Graceful failure if LLM, Chroma, MinIO, SMTP or DB is unavailable.
- No partial success presented as a completed analysis.
- Persist analysis status: queued/running/completed/failed.

22.2 Resource efficiency

- Reuse embedding model and provider clients.
- Use DB connection pools.
- Limit selected-document count for Compare/Analyze.
- Bound top_k and related-document counts.
- Avoid sending an entire department corpus to the LLM.
- Cache safe repeatable metadata/catalog results where useful.

22.3 Scale later, based on evidence

MVP remains Docker Compose + FastAPI + MySQL + MinIO + Chroma. If measured load requires it, the modular design can later move toward multiple API workers, managed MySQL, S3-compatible storage, Qdrant, Redis/job workers and load balancing. Do not introduce those systems before a real trigger.

23. Alembic Migration Strategy

Do not manually mutate production schema as the primary workflow. Make small, reversible and testable Alembic migrations.

Migration	Changes
M1	roles + user_roles
M2	email_otps + admin_allowlist

Migration	Changes
M3	knowledge_bases.scope + is_system_managed
M4	departments
M5	academic_document_metadata
M6	bookmarks
M7	analysis_runs + analysis_documents + analysis_evidence

23.1 Migration safety rules

- Back up current DB before first migration.
- Test upgrade and downgrade in a disposable environment.
- Backfill existing knowledge bases as private unless explicitly designated otherwise.
- Create the system-owned public KB only after RBAC and system-account rules are in place.
- Do not delete existing data merely to simplify migration.

24. End-to-End Implementation Phases

Codex should execute phases sequentially and verify each before proceeding.

Phase	Definition of done
Phase 0 - Protect baseline	Confirm branch, run tests, verify current PDF -> retrieval -> LLM -> cited answer path, create a checkpoint commit.
Phase 1 - Schema foundation	Add roles, user_roles, email_otps, admin_allowlist, KB scope fields, departments, academic metadata, bookmarks.
Phase 2 - Authorization core	Implement current-user resolution, role checks, system owner, KB/document access helpers.
Phase 3 - SMTP OTP	Request, send, verify OTP; create secure session; logout; /auth/me.
Phase 4 - Public corpus	Create the UAF Research Corpus as system-owned public KB.
Phase 5 - Academic metadata	Wire departments, titles, authors, types, years, keywords and validation.
Phase 6 - Public Library	Catalog API, search, filters, pagination, bookmarks, normal researcher read-only experience.
Phase 7 - Admin Public Library	Upload, metadata entry/edit, processing states, retry, publish/archive/delete controls.
Phase 8 - Private Library	Map existing user-owned KB functionality into the Private Library UI and APIs.
Phase 9 - Scope-aware retrieval	Public / Private / Both authorization and retrieval constraints.
Phase 10 - Research Assistant	Reuse chat pipeline with ResearchIQ source scopes and hardened citations.
Phase 11 - Document-scoped retrieval	Exact document filtering required for

Phase	Definition of done
	Summarize/Compare/Analyze.
Phase 12 - Summarize	Structured single-document summary with citations.
Phase 13 - Insights UI	Search/filter/select documents, Compare and Analyze actions.
Phase 14 - Compare	Per-document evidence retrieval + structured comparison JSON + React table.
Phase 15 - Analyze	Research profiles + related-document discovery + bounded cross-study synthesis.
Phase 16 - Contradictions	Supporting/contradicting/qualifying evidence classification and interpretation.
Phase 17 - Candidate gaps	Corpus-qualified gap patterns with citations and careful wording.
Phase 18 - Persistence/history	Store analysis runs, selected docs, evidence snapshots and results.
Phase 19 - Security testing	OTP abuse, RBAC bypass, private ID guessing, prompt injection, upload abuse, sanitization.
Phase 20 - RAG evaluation	Retrieval relevance, citation correctness, faithfulness, abstention, comparison/analysis quality, latency.
Phase 21 - Deployment hardening	HTTPS, secrets, backups, health checks, rate limits, logging, internal-only data services.

25. Acceptance and Evaluation Criteria

25.1 Researcher acceptance scenario

9. Request OTP and sign in successfully.
10. Enter Public Library and filter institutional research.
11. Bookmark a public paper.
12. Upload a private paper and observe Processing -> Ready.
13. Ask a question using Public + Private scope and receive evidence-backed citations.
14. Select two permitted documents and receive a structured comparison table.
15. Run Analyze and receive summary, insights, limitations, contradictions and candidate gaps with citations.
16. Reopen the saved analysis/history and inspect its evidence.

25.2 Admin acceptance scenario

17. Professor authenticates with OTP.
18. Exact email is present in admin_allowlist and Admin role is resolved.
19. Public Library shows management controls.
20. Admin uploads a document with academic metadata.
21. Processing completes and the document becomes discoverable to normal researchers.
22. Normal researcher cannot call the same management endpoints.

25.3 RAG quality metrics

Metric	Expectation
Retrieval relevance	Relevant chunks/documents appear for representative research queries.
Citation correctness	Citations resolve to the actual cited document/page/chunk.
Faithfulness	Grounded claims are supported by retrieved evidence.
Abstention	System reports insufficient evidence when needed.
Comparison integrity	Each comparison cell is derived from the correct source document.
Gap/contradiction support	Every reported pattern links to evidence; wording remains qualified.
Latency	Interactive tasks remain usable; longer analyses expose progress/status.

26. Codex Execution Contract

Paste this PDF into Codex as the governing implementation specification. Codex should not reinterpret the architecture into a new parallel system.

Codex primary instruction
Inspect the existing implementation first and extend it incrementally. Preserve working ingestion, MinIO, Chroma, document processing, chat, streaming and citation behavior unless a documented change explicitly requires modification. Use the existing MySQL schema as the base and make schema changes only through Alembic.

26.1 Mandatory working rules

- Before each phase, inspect the relevant existing models, schemas, routes, services and tests.
- Reuse existing functionality instead of duplicating it.
- Do not build a second RAG service, a second frontend or a second relational database.
- Do not implement authorization only in the frontend.
- Retrieval must be permission-scoped before evidence reaches the LLM.
- Do not begin Compare/Analyze until Public/Private access control, academic metadata and document-scoped retrieval work and are tested.
- Use structured Pydantic response models for AI workflows.
- Validate LLM evidence references against the evidence set before returning results.
- Run tests and verify the affected end-to-end behavior after each phase.
- Commit stable checkpoints before major architectural changes.

26.2 Definition of architectural failure

- Creating a parallel RAG backend instead of extending the existing one.
- Allowing frontend role checks to be the only security boundary.
- Passing unauthorized private chunks to the model.
- Letting the model invent citations or bibliography fields.
- Implementing Analyze as one enormous unbounded prompt over an entire department corpus.

- Hard-coding professor/admin emails in frontend code.
- Mutating the database manually without migrations.

Appendix A. Proposed Schema

A.1 Existing tables retained

```
users
knowledge_bases
documents
document_uploads
document_chunks
processing_tasks
chats
messages
chat_knowledge_bases
api_keys
alembic_version
```

A.2 Existing tables modified

```
users
  hashed_password -> nullable

knowledge_bases
  + scope ENUM/string: private | public
  + is_system_managed boolean
```

A.3 New tables

Table	Suggested fields
roles	id, name
user_roles	user_id, role_id
email_otps	id, email, otp_hash, purpose, attempt_count, expires_at, used_at, created_at
admin_allowlist	id, email, department_id, is_active, added_by, created_at, updated_at
departments	id, name, code, is_active, created_at, updated_at
academic_document_metadata	id, document_id, title, authors_text, abstract, department_id, document_type, publication_year, publication_date, supervisor, keywords, doi, language, timestamps
bookmarks	id, user_id, document_id, created_at; unique(user_id,document_id)
analysis_runs	id, user_id, analysis_type, status, scope, request_json, result_json, timestamps, error_message
analysis_documents	analysis_id, document_id, role
analysis_evidence	id, analysis_id, document_id, chunk_id, page, excerpt, retrieval_rank, retrieval_score, evidence_hash

Appendix B. Core API Contracts

B.1 Authentication

```
POST /api/v1/auth/request-otp
POST /api/v1/auth/verify-otp
POST /api/v1/auth/logout
GET  /api/v1/auth/me
```

B.2 Catalog / Library

```
GET    /api/v1/documents
GET    /api/v1/documents/{id}
POST   /api/v1/bookmarks/{document_id}
DELETE /api/v1/bookmarks/{document_id}
GET    /api/v1/bookmarks
```

B.3 Admin corpus

```
POST   /api/v1/admin/documents
PATCH  /api/v1/admin/documents/{id}/metadata
POST   /api/v1/admin/documents/{id}/retry
POST   /api/v1/admin/documents/{id}/publish
POST   /api/v1/admin/documents/{id}/archive
DELETE /api/v1/admin/documents/{id}
```

B.4 Research / Insights

```
POST /api/v1/research/ask
POST /api/v1/research/summarize
POST /api/v1/insights/compare
POST /api/v1/insights/analyze
GET  /api/v1/analyses/{id}
GET  /api/v1/analyses
```

B.5 Compare request example

```
{
  "document_ids": [81, 96],
  "scope": "both"
}
```

B.6 Analyze request example

```
{
  "document_ids": [81, 96, 112],
  "scope": "both",
  "analysis_types": ["summary", "limitations", "contradictions", "candidate_gaps"]
}
```

Appendix C. Threat and Failure Checklist

Area	Test / failure condition	Expected behavior
OTP	Repeated wrong codes	Attempt counter increments; lock/rate limit after threshold.
OTP	Expired or already-used code	Reject without creating session.
RBAC	Researcher invokes admin upload	403/denied server-side.
Private data	User A requests User B document ID	No metadata/chunks disclosed; request denied.

Area	Test / failure condition	Expected behavior
Prompt injection	PDF tells model to reveal secrets	Treated as document text; system policy unchanged.
Citation	Model references nonexistent evidence ID	Backend rejects/repairs response; never shows invented source.
Upload	Oversized/malformed/unsafe file	Reject safely; no execution; useful error state.
Chroma	Vector service unavailable	Fail gracefully; mark task/request failed, do not fabricate answer.
LLM	Provider unavailable/timeout	Return recoverable error; preserve analysis state.
SMTP	Mail provider unavailable	OTP request reports temporary delivery failure; no false success if code cannot be delivered.
DB	Transaction failure	Rollback related changes; avoid half-created records.
Delete	Public document deletion	Clean/coordinate relational, MinIO and vector state; audit action.
XSS	LLM or document outputs HTML/script	Sanitize before rendering.
Abuse	Huge Analyze selection	Reject above configured document/evidence limits.
Evidence	Low retrieval support	Return insufficient evidence / qualified language.

Final architecture summary

SMTP OTP + explicit RBAC + system-owned Public Library + user-owned Private Library + academic metadata + bookmarks + secure RAG + structured Compare/Analyze + verified citations + evidence-qualified contradictions/gaps. All of it extends the existing RAG Web UI stack instead of replacing it.

Phase 12 - Researcher Portfolio and Publication Requests

Approved phase insertion and closure | Original plan version 1.0, 15 September 2026

This dated amendment governs phase numbering and status wherever the original 15 September 2026 plan conflicts with it. The preserved dated original remains unchanged. The working plan retains its original 23 pages and appends this controlling amendment.

Phase 12 scope

- Add an owner-controlled researcher profile with required name, department, degree and research area; optional student identifiers and photo remain private.

- Allow a researcher to request publication of one processed private paper. Administrators review only that exact requested document, edit request metadata, and approve or deny with a reason.

- Approval indexes a separate public copy before publication; the private original remains private. Per-request consent controls the limited public researcher card. Approved withdrawal archives the public copy and removes it from discovery.

- Use existing authentication, authorization, storage, processing, catalog and retrieval lifecycles. Keep generic private document routes owner-only.

Closure evidence

Implementation and synthetic acceptance completed 16 October 2026. Alembic revision riq_006 passed disposable-MySQL upgrade, downgrade-guard and populated-baseline preservation tests before live application and backup. Backend regression: 331 passed / 41 skipped; disposable MySQL: 25 passed; frontend: 80 passed; TypeScript and production build passed.

Synthetic live acceptance covered profile/photo privacy, ownership, request-scoped review, separate public indexing, preserved private source, consent projection, PDF open/download, retrieval, cited streaming, saved-chat reopening and withdrawal removal. Synthetic records were removed and baseline totals matched. Implementation recovery tag: researchiq-phase12-recovery-20261016.

Status

Phase 12 is formally closed by this amendment. No historical phase number, commit or tag is rewritten. Phase 13 is the next implementation gate; complete and checkpoint it before starting Phase 14.

Amended Sequential Roadmap: Phases 12-22

Completed Phases 0-11 and all historical checkpoints remain unchanged

The approved Phase 12 insertion occupies the previous summary gate. Original planned Phases 12-21 shift forward by one; use the phase numbers and gate order below for all future work.

Phase 12 - Researcher Portfolio and Publication Requests (complete)

Profiles, publication and withdrawal requests, scoped review, consent-limited public cards; closed by this amendment.

Phase 13 - Summarize (original Phase 12)

Structured single-document summary with citations, reusing exact document-scoped retrieval.

Phase 14 - Insights UI (original Phase 13)

Search, filter and select permitted documents; expose Compare and Analyze actions.

Phase 15 - Compare (original Phase 14)

Per-document evidence retrieval, structured comparison output and an accessible React table.

Phase 16 - Analyze Research (original Phase 15)

Profiles, related-document discovery and bounded cross-study synthesis.

Phase 17 - Contradictions (original Phase 16)

Classify and interpret supporting, contradicting and qualifying evidence.

Phase 18 - Candidate Gaps (original Phase 17)

Corpus-qualified gap patterns, evidence citations and careful wording.

Phase 19 - Persistence and History (original Phase 18)

Store analysis runs, selected documents, evidence snapshots and results.

Phase 20 - Cross-feature Security (original Phase 19)

Test OTP abuse, RBAC bypass, private ID guessing, prompt injection, upload abuse and sanitization.

Phase 21 - RAG Evaluation (original Phase 20)

Evaluate retrieval relevance, citations, faithfulness, abstention, quality and latency.

Phase 22 - Deployment Hardening (original Phase 21)

Harden HTTPS, secrets, backups, health checks, rate limits, logging and internal-only data services.

Execution rule

Work incrementally. Finish implementation, tests, acceptance and a recovery checkpoint for one phase before beginning the next. This amendment does not authorize skipping any gate.

Phase 13 - Structured Single-Document Summary

Implementation contract and acceptance gate

Purpose and entry points

Provide a temporary, structured academic summary of one selected paper. Add a Summarize action to the existing document-detail view in both Private Library and Public Library. Chat remains unchanged; saved summary history is deferred to Phase 19.

API and authorization

Add POST /api/retrieval/documents/summarize with a strict body containing document_id (positive integer) and scope (public or private; no Both scope). Reject unknown fields. Private scope requires an active requester who owns the ready document. Public scope requires a currently published document in the system-managed corpus. Return the same unavailable response for missing and unauthorized IDs.

Reuse exact Phase 11 document-scoped retrieval and its fresh SQL authorization/readiness checks. Every retrieval call must include only the selected document ID and requested scope. Recheck permission and publication/readiness after provider generation before returning output.

Evidence and summary shape

Return five sections: objective/research question; methods, design and context; key findings; limitations; and implications/conclusion. Represent supported content as short claim-level statements. Each claim cites one or more IDs from the retrieved evidence set. Resolve every ID server-side into the selected document, its filename and its page; reject malformed output, uncited claims and any unknown or out-of-scope citation.

Treat all document text as untrusted data and explicitly ignore instructions embedded in it. If a section has no supporting evidence, display: Not identified in retrieved evidence. If retrieval yields no usable evidence, return a clear insufficient-evidence state without calling the model. Keep results temporary: no migration, persisted analysis, history or new RAG store.

Provider and failure handling

Use the configured free-model provider, JSON-object generation and the existing bounded pre-output 429 policy: at most one retry, honoring Retry-After only up to five seconds. Do not use a paid fallback. Return a recoverable error on provider, parsing, retrieval or citation-validation failure; never show unvalidated partial output.

Acceptance gate

- Backend tests cover private owner/foreign user, public published/unpublished, ready/processing/failed, exact-document selection, empty evidence, prompt injection, malformed JSON and invalid citation IDs.

- Test a single bounded 429 retry and persistent throttling; validate citation metadata and page links.

- Frontend tests cover both library entry points, loading/error/retry, all sections, unavailable markers, accessible citation links and narrow viewport behavior.

- Run backend regression, frontend tests, TypeScript and production build. Use disposable synthetic public/private PDFs for live acceptance; verify owner boundaries, PDF/page links and generation, then remove only the fixtures.

- After passing acceptance, record the Phase 13 receipt, create a recovery commit and dated annotated tag. Do not start Phase 14 before that checkpoint.